The Agent Playbook

A repeatable process for building trustworthy Copilot agents · Data Science & AI Team
Distilled from the IM Residency coaching engagement — a template-driven summarization agent now in production use — and validated licensing and testing work.

> **The premise: the agent is the last 15 minutes of the work.** Everything that makes it trustworthy happens before the builder is opened and after the first version ships.

1 Capture the human process first

Before any AI: what do the humans do today, and what artifacts already exist? Meeting structures, templates, checklists, report formats, review steps. If the process isn't written down, write it down now — an agent built on a vague process automates the vagueness. (In the coaching engagement, the session framework, key questions, and checklist existed on paper first; the agent inherited them.)

2 Design the output before the agent

Produce the exact template of what good output looks like — headings, tables, tone, what's included and what's deliberately excluded — and get the owner to approve *that document* before writing any instructions. Two outputs for two audiences (e.g., a detailed internal version and a concise recipient-facing version)? Decide now, and decide which fields must match **verbatim** between them so they can't drift.

3 Write the instructions

Structure every instruction block the same way:

1. **Role and input** — what the agent is, what the user will give it, and what to do if the input is missing ("ask for it and do nothing else").
2. **Evidence rules** — use ONLY what's in the input; never infer or embellish; unclear → leave it out; unanswered → "Not discussed"; never fill gaps from context. These rules do more for trustworthiness than anything else in the block.
3. **Attribution and tone rules** — whose words are whose; descriptive, not evaluative.
4. **The output template, embedded in full** — section by section, including any closed sets (checklist items, status values) listed explicitly. The template lives *in the instructions*, not in a knowledge source (Step 4).
5. **Boundary rules** — what goes in which output; content that fits nowhere goes to a "Notes" section rather than being dropped or shoehorned.

6. **A closing line** the agent appends, reminding the user who reviews what.

Keep the whole block under the builder's instruction limit (~8,000 characters); closed sets and terse section specs beat prose.

4 Default to NO knowledge sources

Templates and rules go in the instructions; per-task documents get uploaded by the user in the conversation. This keeps the agent usable by unlicensed colleagues without premium-knowledge licensing complications, keeps behavior deterministic (nothing changes because a SharePoint file moved), and keeps the agent portable. Add knowledge sources only when the agent genuinely must consult a living document corpus — and treat that as a licensing conversation, not just a checkbox.

5 Build in the Configure tab

Copilot Chat → Agents → Create agent → **Configure** (skip "Describe" — we paste exact instructions, we don't negotiate them with a wizard). Name with a version suffix (`… v1`), short description, paste instructions, knowledge off, capabilities off, two conversation starters (one that does the work, one that explains what the agent needs).

6 Test with synthetic traps before real data

Author a synthetic input with **deliberate traps**, then check the output against them:

- **An omission trap:** something the template expects that the input deliberately lacks → must be reported "Not covered"/"Not discussed," never invented.
- **A boundary trap:** content that belongs in one output and not another (e.g., personal context → internal notes only) → must land on the right side of the line.
- **A verbatim trap:** wording that must survive word-for-word across outputs → must not be paraphrased.

An agent that passes planted traps earns a trial on real data; one that confidently fills gaps fails the build. Keep the synthetic file — it's the regression test for every future version. Never use real data for testing, demos, or licensing checks.

7 Share deliberately, and tell users where it works

Share to **specific people**, not the whole org, until steady state. Tell non-premium users explicitly: the agent lives in the **M365 Copilot app / m365.cloud.microsoft**, not inside Word. (Validated in our tenant: unlicensed users can run shared agents end to end, including file upload.)

8 Version loop, because agents don't learn

Mid-conversation corrections vanish. The loop that actually improves an agent: one collection point for user feedback (Teams thread or doc) → fold into revised instructions → update agent, bump name to vN+1 → rerun the synthetic trap test → repeat until steady. The owner of the process owns the feedback; we own folding it in (or teach them to).

9 Keep a human gate

Agent output is a draft until a named human reviews it. Where the output faces a third party (a learner, a patient-adjacent record, a committee), add a second gate: the affected person confirms accuracy before the document is finalized. This isn't overhead — in our experience it's what makes process owners comfortable deploying at all.

> **What this playbook deliberately avoids:** no custom UIs, no hosted services, no databases, nothing the AI team must operate long-term. Everything above is documents, instructions, and M365-native configuration owned by the process owner after handoff. If a request genuinely can't fit that shape, it's not an agent request — it's a project, and it gets scoped as one.
